Univerzitet u Zenici
Politehnički fakultet

BILJEŠKE S PREDAVANJA · SEDMICA 4

Mrežna konfiguracija i firewall kao kod

Mreža kao kod: segmenti i usmjeravanje (`VPC/subnet`, routing, NAT), vatrozid pravilima u `nftables`, segmentacija web/app/db, bastion host, WireGuard VPN te uvod u bridge i VLAN. Pravila se opisuju datotekama i podacima, a ishod se provjerava testom povezivosti.

PREDMET	Računarstvo u oblaku
STUDIJ	II ciklus studija, III semestar, akademska godina 2026/27
NASTAVNIK	V. prof. dr. Đulaga Hadžić
ASISTENT	ass. Mr. Adin Jahić
TERMIN	petak, 06.11.2026, 17:15–20:15, Kabinet 3204
TEMA	Mrežna konfiguracija i firewall kao kod
KOD	github.com/Dadyze/RuO-2026-27 (mape `s4/p1` do `s4/p5`)

OKVIR NASTAVNE JEDINICE

Ciljevi i način rada

U prethodne tri sedmice su mašine i servisi opisani kodom. Ostaje pitanje *ko s kim smije razgovarati*. Mreža i vatrozid su najčešći izvor grešaka i sigurnosnih propusta, a ručno podešavanje je teško ponoviti i još teže pregledati. Zato se i oni pišu kao kod: pravila su u datoteci, datoteka je u Gitu, a ishod se provjerava testom, ne pogađanjem.

Za rad nije potreban račun u oblaku. Mrežni elementi (ruter, segmenti, tunel) prave se od **Linux network namespace-ova**, istog mehanizma na kojem se zasnivaju kontejneri iz prve sedmice. Jedan namespace je jedna izolirana mreža sa svojim sučeljima, adresama i tabelom usmjeravanja, pa tako radi i svaka mašina u labu.

Nakon nastavne jedinice student je u stanju:

- povezati pojmove oblaka (VPC, subnet, route table, NAT, security group) s Linux alatima koji ih ostvaruju,
- napraviti segmentiranu mrežu s ruterom i NAT-om i objasniti put paketa kroz nju,
- napisati stateful vatrozid s politikom "sve zabranjeno, dozvole se nabrajaju" u `nftables` i provjeriti ga matricom povezivosti,
- postaviti WireGuard VPN i ograničiti šta klijent kroz njega smije dosegnuti,

- generisati pravila vatrozida iz podataka Ansible rolom i provjeriti idempotentnost,
- objasniti ulogu bastion hosta i podesiti SSH klijent da ga koristi (`ProxyJump`),
- objasniti razliku između L2 (bridge, VLAN) i L3 (ruter) segmentacije.

Okruženje i pokretanje

Alate za ovu sedmicu (`ip`, `nft`, `wg`) instalira `00-setup.sh`. Ako je već pokrenuta ranije, ponovo se pokrene (idempotentna je), pa se primjeri pokreću redom:

```bash
cd ~/RuO-2026-27 && git pull
./00-setup.sh                    # dodaje iproute2, nftables, wireguard-tools
make check                       # da li su nft i wg prisutni
make s4-p1                       # zatim s4-p2 ... s4-p5; opcionalni koraci imaju sufiks -up /
-apply
```

RAD BEZ TROŠKOVA: PREGLED UMJESTO IZVRŠAVANJA

Osnovni cilj svakog primjera samo pokazuje šta bi se desilo: `DRY=1` ispisuje naredbe bez izvršavanja, `nft -c` provjerava sintaksu bez učitavanja, a `ansible-playbook --check --diff` prikazuje razliku. Stvarno izvršavanje ima sufiks `-up` ili `-apply`, traži `sudo` i mijenja samo mrežu u izoliranim namespace-ovima, ne mrežu računara. Sve se briše sa `make s4-clean`.

VAŽNO

Primjeri `-up` kreiraju namespace-ove i učitavaju pravila, pa traže administratorska prava. Redoslijed je obavezan: `s4-p1-up` (lab), zatim `s4-p2-apply` (vatrozid), zatim `s4-p3-up` (VPN). WireGuard kernelski modul na nekim sistemima (npr. WSL2) ne postoji; skripta tada sama koristi korisnički `wireguard-go`.

---

1 · POJAM I POZADINA

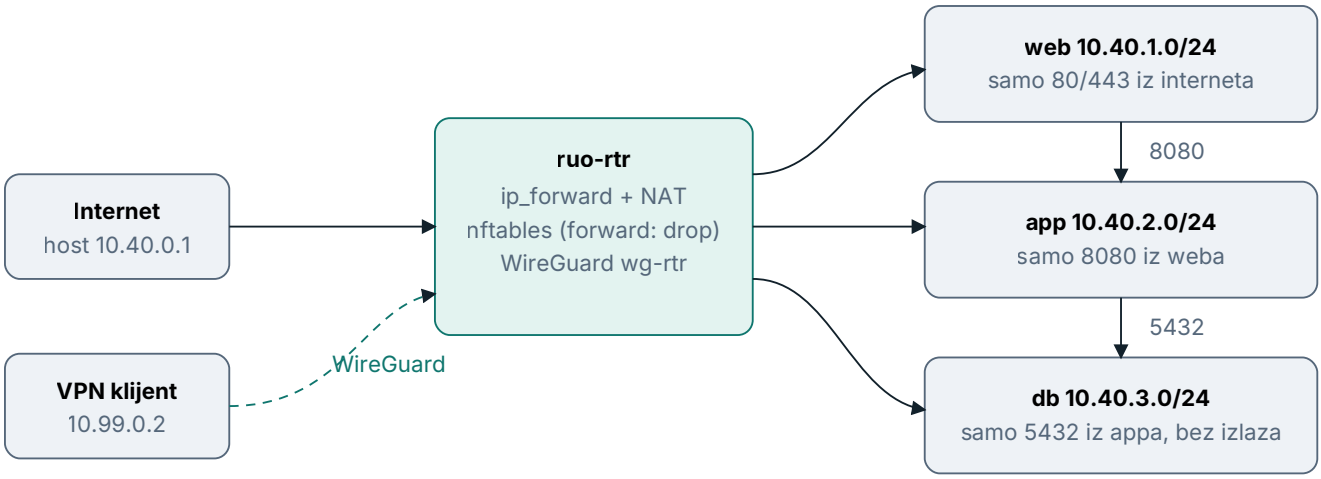
Od mašina do mreže

Mreža u oblaku se sastoji od nekoliko elemenata koje svaki pružatelj (AWS, Azure, GCP) naziva drugačije, ali rade isto. Na Linuxu svaki od njih ima konkretnu zamjenu, pa se isto razmišljanje može vježbati lokalno i besplatno.

OBLAK (AWS / AZURE)	ZNAČENJE	U OVOM LABU
VPC / VNet	izolirana privatna mreža	ruter `ruo-rtr` i namespace-ovi iza njega
Subnet	podmreža unutar VPC-a	segment `10.40.N.0/24` (namespace + veth par)
Route table	kuda ide koji promet	`ip route`, zadani put preko rutera
Internet / NAT gateway	izlaz privatnih mašina prema internetu	uplink `r-up` + `masquerade`
Security group / NSG	stateful pravila uz mašinu ili podmrežu	`nftables` lanac `forward`, zadano `drop`
Bastion host	jedina ulazna tačka za administraciju	jedna adresa s dozvolom za SSH + `ProxyJump`
VPN gateway	siguran tunel do vanjske mreže	WireGuard (`wg-rtr`)
VLAN / L2 preklopnik	razdvajanje prometa na sloju 2	Linux `bridge` + 802.1Q

POZADINA: NFTABLES

Linux vatrozid je dugo bio `iptables` (više odvojenih alata za IPv4, IPv6, ARP). `nftables` ga je zamijenio 2014. godine (jezgra 3.13): jedan alat, jedan jezik, skupovi (*sets*) i atomsko učitavanje cijele datoteke, a od Debiana 10 je i zadani. Pravila su organizovana u **tabele** (`table`), tabele u **lance** (`chain`), a lanci se vežu na *hook* u putu paketa kroz jezgru: `input` (paket za sam ruter), `forward` (paket koji prolazi kroz ruter), `output` i `prerouting` / `postrouting` (NAT). Praćenje veza (*connection tracking*, `ct`) pamti stanje, pa vatrozid dozvoljava odgovore na dozvoljene veze bez posebnog pravila.



Slika 1. Topologija laba: sav promet između segmenata prolazi kroz ruter, pa se politika provodi na jednom mjestu. Strelice između segmenata su jedini dozvoljeni putevi.

ZAŠTO SEGMENTACIJA

Ako napadač preuzme web server, u ravnoj mreži odmah vidi i bazu. Segmentacijom je web smije dosegnuti samo kroz aplikacijski port, a bazu uopće ne. Politika "sve zabranjeno, dozvole se nabrajaju" (*default deny*) znači da zaboravljeno pravilo uzrokuje prekid veze koji se odmah primijeti, a ne tihu rupu.

2 · PRIMJER 1 · S4/P1

Segmenti, ruter i NAT od namespace-ova

> **POZADINA PRIMJERA**
> Svaki segment je namespace (`ruo-web` , `ruo-app` , `ruo-db`) spojen na ruter `ruo-rtr` **veth parom**: virtualnim kablom čiji se jedan kraj nalazi u segmentu, a drugi u ruteru. Ruter ima po jedno sučelje u svakom segmentu (`.1` u podmreži) i uplink prema "internetu" (host). Mašine u segmentu imaju zadani put (`default via`) preko rutera. Bez `net.ipv4.ip_forward=1` ruter ne bi prosljeđivao pakete, što je čest razlog zašto "sve je spojeno, a ništa ne radi".

bash — Isječak 1 · s4/p1/lab.sh (izvod)

```
  # 1. ruter i uplink prema "internetu" (host)
  run ip netns add ruo-rtr
  run ip link add h-up type veth peer name r-up netns ruo-rtr
  run ip addr add 10.40.0.1/30 dev h-up
  run ip link set h-up up
  run ip -n ruo-rtr addr add 10.40.0.2/30 dev r-up
  run ip -n ruo-rtr link set lo up
  run ip -n ruo-rtr link set r-up up
  run ip route add 10.40.0.0/16 via 10.40.0.2          # povratni put do segmenata
  rtr sysctl -qw net.ipv4.ip_forward=1                 # ruter prosljeđuje pakete
  run ip -n ruo-rtr route add default via 10.40.0.1

  # 2. segmenti: namespace + veth par prema ruteru + zadani put preko rutera
  for s in $SEGMENTI; do
    ime="${s%%:*}"; n="${s##*:}"
    run ip netns add "ruo-$ime"
    run ip link add "r-$ime" netns ruo-rtr type veth peer name eth0 netns "ruo-$ime"
    run ip -n ruo-rtr addr add "10.40.$n.1/24" dev "r-$ime"
    run ip -n ruo-rtr link set "r-$ime" up
    run ip -n "ruo-$ime" addr add "10.40.$n.2/24" dev eth0
    run ip -n "ruo-$ime" link set lo up
    run ip -n "ruo-$ime" link set eth0 up
    run ip -n "ruo-$ime" route add default via "10.40.$n.1"
  done
```

bash — Isječak 2 · pokretanje

```
make s4-p1          # DRY=1: ispisuje naredbe, ništa ne mijenja
make s4-p1-up       # opcionalno: stvarno pravi ruter i tri segmenta (traži sudo)
```

izlaz make s4-p1-up: stanje rutera (skraćeno, imena sučelja se mogu razlikovati)

```
== ruo-rtr
r-up@if16        UP             10.40.0.2/30
r-web@if2        UP             10.40.1.1/24
r-app@if2        UP             10.40.2.1/24
r-db@if2         UP             10.40.3.1/24
default via 10.40.0.1 dev r-up
10.40.0.0/30 dev r-up proto kernel scope link src 10.40.0.2
10.40.1.0/24 dev r-web proto kernel scope link src 10.40.1.1
10.40.2.0/24 dev r-app proto kernel scope link src 10.40.2.1
10.40.3.0/24 dev r-db proto kernel scope link src 10.40.3.1
```

Ruter ima put do svake podmreže jer su mu sučelja direktno u njima (`proto kernel scope link`), a sve ostalo ide na `default via 10.40.0.1`. Za povratni put host mora znati da je `10.40.0.0/16` iza rutera; to je jedan red u skripti (`ip route add 10.40.0.0/16 via 10.40.0.2`). Isti princip vrijedi u oblaku: odgovor mora imati put nazad, a u tabeli usmjeravanja je to *route table* podmreže.

NAT

Privatne adrese (`10.0.0.0/8`) se ne usmjeravaju po internetu. **Masquerade** mijenja izvornu adresu paketa u adresu izlaznog sučelja rutera i pamti prevod, pa se odgovor vraća pravoj mašini. To je *SNAT* kakav radi kućni ruter i NAT gateway u oblaku. Pravilo je u lancu `postrouting` (Isječak 3).

---

3 · PRIMJER 2 · S4/P2

Vatrozid u nftables: default deny i segmentacija

POZADINA PRIMJERA

Pravila se učitavaju u ruter, pa vrijede za sav promet između segmenata. Lanac `forward` ima `policy drop`: sve što nijedno pravilo ne dozvoljava pada. Prvo pravilo (`ct state established,related accept`) propušta odgovore na dozvoljene veze, pa je dovoljno opisati samo *smjer uspostave* veze (stateful vatrozid, kao security group u oblaku). Datoteka počinje s `table inet ruo` i `delete table inet ruo`: prvi red osigurava da tabela postoji (inače bi brisanje javilo grešku), a drugi je briše, pa učitavanje uvijek počinje od čistog stanja i može se ponavljati (**idempotentnost**). Cijela datoteka se učitava atomski: ili sva pravila, ili nijedno.

nft — Isječak 3 · s4/p2/firewall.nft

```
#!/usr/sbin/nft -f
# s4/p2: segmentacija web/app/db na ruteru (ruo-rtr). Učitava se atomski: cijela datoteka ili ništa.
# Idiom "table ...; delete table ...; table ... { }" čini učitavanje idempotentnim bez diranja tuđih tabela.

define web    = 10.40.1.0/24
define app    = 10.40.2.0/24
define db     = 10.40.3.0/24
define uplink = "r-up"
define vpn_if = "wg-rtr"                        # WireGuard sučelje (s4/p3)

table inet ruo
delete table inet ruo
```

```
table inet ruo {
  set web_ports { type inet_service; elements = { 80, 443 } }

  # promet KROZ ruter (između segmenata): zadano je zabrana, dozvole se nabrajaju
  chain forward {
    type filter hook forward priority filter; policy drop;

    ct state established,related accept          # odgovori na dozvoljene veze
    ct state invalid drop

    iifname $uplink ip daddr $web tcp dport @web_ports accept   # internet -> web: samo 80/443
    ip saddr $web ip daddr $app tcp dport 8080 accept
# web -> app: samo aplikacijski port
    ip saddr $app ip daddr $db  tcp dport 5432 accept           # app -> db: samo baza
    ip saddr { $web, $app } oifname $uplink accept              # izlaz prema internetu: web i
app, ne db
    iifname $vpn_if ip daddr $app tcp dport { 22, 8080 } accept # VPN klijent -> app (SSH,
aplikacija), ne db

    limit rate 5/minute log prefix "ruo-drop: "                 # sve ostalo pada (policy drop)
  }

  # promet KA samom ruteru
  chain input {
    type filter hook input priority filter; policy drop;
    iifname "lo" accept
    ct state established,related accept
    udp dport 51820 accept                                      # WireGuard handshake (s4/p3)
    icmp type echo-request limit rate 5/second accept
  }

  # NAT: segmenti izlaze prema uplinku s adresom rutera
  chain postrouting {
    type nat hook postrouting priority srcnat;
    oifname $uplink ip saddr 10.40.0.0/16 counter masquerade
  }
}
```

POJAM	ZNAČENJE
`table inet ruo`	kontejner za lance; `inet` pokriva IPv4 i IPv6 jednim skupom pravila
`chain ... hook forward`	lanac vezan na tačku u jezgri; `priority` određuje redoslijed među lancima na istom hooku
`policy drop`	zadana odluka kad nijedno pravilo ne pogodi paket
`ct state`	stanje veze iz connection trackinga (`established`, `related`, `invalid`)
`set web_ports`	imenovani skup (portova, adresa); mijenja se na jednom mjestu
`define`	varijabla na nivou datoteke, razrješava se pri učitavanju
`masquerade`	NAT koji koristi adresu izlaznog sučelja (lanac `postrouting`)

GRANICA: VIŠE TABELA

Paket mora proći *sve* lance na istom hooku. `accept` u jednoj tabeli završava samo taj lanac, pa druga tabela s `policy drop` i dalje može odbaciti isti paket. Zato se dozvola dodaje u tabelu koja zabranjuje (kao u Primjeru 3), a ne u zasebnu.

bash — Isječak 4 · pokretanje

```
make s4-p2          # nft -c: provjera sintakse bez učitavanja
make s4-p2-apply    # opcionalno: matrica PRIJE pa POSLIJE učitavanja (traži s4-p1-up)
```

Matrica povezivosti (`s4/p2/test.sh`) na svako odredište postavi TCP slušaoca, pa pokuša vezu iz zadanog izvora. Prva kolona je izvor, zatim odredište i port, ishod i očekivanje iz politike. Prije vatrozida je mreža ravna i sve prolazi (šest razlika od politike), a poslije je ishod jednak politici u svih deset slučajeva:

izlaz PRIJE učitavanja firewall.nft

```
  host   -> 10.40.1.2  :80     dozvoljeno  (očekivano: dozvoljeno) ok
  host   -> 10.40.2.2  :8080   dozvoljeno  (očekivano: blokirano ) RAZLIKA
  host   -> 10.40.3.2  :5432   dozvoljeno  (očekivano: blokirano ) RAZLIKA
  web    -> 10.40.2.2  :8080   dozvoljeno  (očekivano: dozvoljeno) ok
  web    -> 10.40.2.2  :22     dozvoljeno  (očekivano: blokirano ) RAZLIKA
  web    -> 10.40.3.2  :5432   dozvoljeno  (očekivano: blokirano ) RAZLIKA
  app    -> 10.40.3.2  :5432   dozvoljeno  (očekivano: dozvoljeno) ok
  app    -> 10.40.3.2  :22     dozvoljeno  (očekivano: blokirano ) RAZLIKA
  web    -> 10.40.0.1  :9000   dozvoljeno  (očekivano: dozvoljeno) ok
  db     -> 10.40.0.1  :9000   dozvoljeno  (očekivano: blokirano ) RAZLIKA
```

izlaz POSLIJE učitavanja firewall.nft

```
  host   -> 10.40.1.2  :80     dozvoljeno  (očekivano: dozvoljeno) ok
  host   -> 10.40.2.2  :8080   blokirano   (očekivano: blokirano ) ok
  host   -> 10.40.3.2  :5432   blokirano   (očekivano: blokirano ) ok
  web    -> 10.40.2.2  :8080   dozvoljeno  (očekivano: dozvoljeno) ok
  web    -> 10.40.2.2  :22     blokirano   (očekivano: blokirano ) ok
  web    -> 10.40.3.2  :5432   blokirano   (očekivano: blokirano ) ok
  app    -> 10.40.3.2  :5432   dozvoljeno  (očekivano: dozvoljeno) ok
  app    -> 10.40.3.2  :22     blokirano   (očekivano: blokirano ) ok
  web    -> 10.40.0.1  :9000   dozvoljeno  (očekivano: dozvoljeno) ok
  db     -> 10.40.0.1  :9000   blokirano   (očekivano: blokirano ) ok
```

nft — Isječak 5 · nft list chain inet ruo forward (stanje u jezgri)

```
table inet ruo {
  chain forward {
    type filter hook forward priority filter; policy drop;
    ct state established,related accept
    ct state invalid drop
    iifname "r-up" ip daddr 10.40.1.0/24 tcp dport @web_ports accept
    ip saddr 10.40.1.0/24 ip daddr 10.40.2.0/24 tcp dport 8080 accept
    ip saddr 10.40.2.0/24 ip daddr 10.40.3.0/24 tcp dport 5432 accept
    ip saddr { 10.40.1.0-10.40.2.255 } oifname "r-up" accept
    iifname "wg-rtr" ip daddr 10.40.2.0/24 tcp dport { 22, 8080 } accept
    limit rate 5/minute burst 5 packets log prefix "ruo-drop: "
  }
}
```

ŠTA JEZGRA RADI S PRAVILIMA

Ispis stanja se razlikuje od izvorne datoteke: `@web_ports` ostaje skup, a lista `{ $web, $app }` je spojena u interval `10.40.1.0-10.40.2.255` jer su podmreže susjedne. Smisao je isti, ali treba znati čitati ispis. Brojači (`counter`) se mogu dodati pravilima da se vidi koliko je paketa pogodilo koje pravilo, što je prvi alat za otkrivanje zašto veza ne radi.

---

4 · PRIMJER 3 · S4/P3

WireGuard VPN: tunel s ograničenim pristupom

POZADINA PRIMJERA

WireGuard je VPN ugrađen u Linux od jezgre 5.6 (2020). Za razliku od starijih rješenja (OpenVPN, IPsec) ima malo koda i malo opcija: svaki učesnik ima **par ključeva**, a konfiguracija je lista *peerova*. Promet ide kroz UDP, a sučelje (`wg-rtr`) se ponaša kao obično mrežno sučelje. Središnji pojam je `allowed-ips`: to je istovremeno *tabela usmjeravanja* (šta se šalje kroz tunel) i *kontrola pristupa* (s kojih adresa se paketi prihvataju od tog peera). Ovdje klijent kroz tunel dobija samo podmrežu `app`; web i db nisu u njegovom `allowed-ips`, pa za njih ne postoji ni put.

bash — Isječak 6 · s4/p3/wg.sh (izvod)

```
  # WireGuard sučelje na ruteru: tunelska mreža 10.99.0.0/24, UDP 51820
  wg_sucelje ruo-rtr wg-rtr
  run ip netns exec ruo-rtr wg set wg-rtr listen-port 51820 private-key "$DIR/rtr.key" \
        peer "$(pub klijent)" allowed-ips 10.99.0.2/32
  run ip -n ruo-rtr addr add 10.99.0.1/24 dev wg-rtr
  run ip -n ruo-rtr link set wg-rtr up

  # klijent: sve za 10.40.2.0/24 (app) ide kroz tunel, ne cijeli internet (split tunnel)
  wg_sucelje "$KLIJENT" wg-kl
  run ip netns exec "$KLIJENT" wg set wg-kl private-key "$DIR/klijent.key" \
        peer "$(pub rtr)" endpoint 10.40.9.1:51820 allowed-ips 10.99.0.1/32,10.40.2.0/24
persistent-keepalive 5
  run ip -n "$KLIJENT" addr add 10.99.0.2/24 dev wg-kl
  run ip -n "$KLIJENT" link set wg-kl up
  run ip -n "$KLIJENT" route add 10.40.2.0/24 dev wg-kl
```

KLJUČEVI

Privatni ključ ne napušta svoju mašinu, a razmjenjuju se samo javni. Skripta ih pravi u `s4/p3/keys/`, koji je u `.gitignore`. Pravi VPN s privatnim ključem u repozitoriju je jedna od najčešćih grešaka; tajne pripadaju Vaultu iz sedmice 3 ili menadžeru tajni.

bash — Isječak 7 · pokretanje

```
make s4-p3          # DRY=1: ispisuje naredbe, ključevi se ne prave
make s4-p3-up       # opcionalno: tunel + test pristupa (traži s4-p1-up i s4-p2-apply)
```

izlaz wg show wg-rtr (ruter, nakon uspostave tunela)

```
== ruo-rtr
interface: wg-rtr
  public key: WsE78C8EYjTlIlSl7eX08VHMmRP5pZe89UF7gYjZL1A=
  private key: (hidden)
  listening port: 51820

peer: 62Qj7dkX50NfblT3FeOd0JrqDcQJq6Aapo6om+liESc=
  endpoint: 10.40.9.2:38238
  allowed ips: 10.99.0.2/32
  latest handshake: 7 seconds ago
  transfer: 212 B received, 92 B sent
```

izlaz s4/p3/test.sh

```
  klijent -> 10.40.2.2  :8080   dozvoljeno  (očekivano: dozvoljeno) ok
  klijent -> 10.40.3.2  :5432   blokirano   (očekivano: blokirano ) ok
  klijent -> 10.40.1.2  :80     blokirano   (očekivano: blokirano ) ok

  promet kroz tunel (ruter): 596 B primljeno, 380 B poslano
```

Tunel se uspostavlja *tek kad prođe prvi promet* (`latest handshake` prije toga nije prikazan), a `persistent-keepalive` održava vezu kroz NAT. Test pokazuje da klijent doseže `app`, ali ne i `db` (nema puta u tunelu ni pravila u vatrozidu) ni `web` (nije u `allowed-ips`), pa su isključena dva nezavisna sloja zaštite.

ŠTA SE DESI AKO SE ZABORAVI PRAVILO

Prije nego što su u `firewall.nft` dodani `udp dport 51820 accept` i pravilo za `wg-rtr`, handshake nije prolazio: klijent je slao, ali u `wg show` nije bilo ni `latest handshake`, ni primljenih bajtova. Vatrozid nije prijavio grešku, paketi su tiho pali (`policy drop`). Tako izgleda najčešći problem s VPN-om, a provjerava se redom: tunel, vatrozid na ulazu (`input`), vatrozid za prosljeđivanje (`forward`), ruta.

---

5 · PRIMJER 4 · S4/P4

Bridge i VLAN: segmentacija na sloju 2

POZADINA PRIMJERA

Do sada je segmentacija bila na sloju 3: ruter razdvaja podmreže i vatrozid odlučuje šta prolazi. **Bridge** je virtualni preklopnik (sloj 2): okvire prosljeđuje između portova prema MAC adresama. **VLAN** (IEEE 802.1Q) dijeli taj preklopnik na više odvojenih L2 domena: okvir nosi 4-bajtnu oznaku s identifikatorom VLAN-a (12 bita, vrijednosti 1 do 4094) i preklopnik ga isporučuje samo portovima istog VLAN-a. *Pristupni* (`access`) port ne vidi oznake: preklopnik sam stavlja neoznačene okvire u VLAN. *Trunk* port nosi više VLAN-ova na istoj žici, pa mašina na drugoj strani razdvaja promet podinterfejsima (`eth0.10`, `eth0.20`). Mašine u različitim VLAN-ovima ne mogu razgovarati bez rutera, što je ista ideja kao segmenti iz Primjera 1, ali ostvarena na sloju 2.

bash — Isječak 8 · s4/p4/vlan.sh (izvod)

```
port() { # ime  vlan  ip      pristupni port: pakete bez oznake sklopka sama stavlja u VLAN
  run ip netns add "ruo-$1"
```

```
  run ip link add "p-$1" netns ruo-sw type veth peer name eth0 netns "ruo-$1"
  sw link set "p-$1" master br0
  sw link set "p-$1" up
  run bridge -n ruo-sw vlan add dev "p-$1" vid "$2" pvid untagged   # ulazni okvir bez oznake -
> VLAN $2
  run bridge -n ruo-sw vlan del dev "p-$1" vid 1                    # ukloni zadani VLAN 1 s
porta
  run ip -n "ruo-$1" addr add "$3" dev eth0
  run ip -n "ruo-$1" link set lo up
  run ip -n "ruo-$1" link set eth0 up
}

gore() {
  run ip netns add ruo-sw
  sw link add br0 type bridge vlan_filtering 1       # bez vlan_filtering bridge ignoriše VLAN
oznake
  sw link set br0 up
  port h10a 10 10.50.10.11/24
  port h10b 10 10.50.10.12/24
  port h20  20 10.50.20.21/24

  # trunk: okviri nose oznaku (tagged); host razdvaja VLAN-ove podinterfejsima eth0.10 i
eth0.20
  run ip netns add ruo-trk
  run ip link add p-trk netns ruo-sw type veth peer name eth0 netns ruo-trk
  sw link set p-trk master br0
  sw link set p-trk up
  run bridge -n ruo-sw vlan add dev p-trk vid 10
  run bridge -n ruo-sw vlan add dev p-trk vid 20
  run bridge -n ruo-sw vlan del dev p-trk vid 1
  run ip -n ruo-trk link set lo up
  run ip -n ruo-trk link set eth0 up
  run ip -n ruo-trk link add link eth0 name eth0.10 type vlan id 10
  run ip -n ruo-trk link add link eth0 name eth0.20 type vlan id 20
  run ip -n ruo-trk addr add 10.50.10.1/24 dev eth0.10
  run ip -n ruo-trk addr add 10.50.20.1/24 dev eth0.20
  run ip -n ruo-trk link set eth0.10 up
  run ip -n ruo-trk link set eth0.20 up
}
```

IZVOR	ODREDIŠTE	OČEKIVANO	ZAŠTO
h10a	h10b (VLAN 10)	dozvoljeno	isti VLAN, ista L2 domena
h10a	h20 (VLAN 20)	blokirano	drugi VLAN, nema L2 puta ni rutera
h10a	trunk host, adresa u VLAN-u 10	dozvoljeno	trunk nosi oznaku 10
h20	trunk host, adresa u VLAN-u 20	dozvoljeno	trunk nosi i oznaku 20 na istoj žici
h10a	trunk host, adresa u VLAN-u 20	blokirano	slušalac postoji, ali je u drugom VLAN-u

bash — Isječak 9 · pokretanje

```
make s4-p4          # DRY=1: ispisuje naredbe
make s4-p4-up       # opcionalno: bridge, VLAN-ovi i test izolacije
make s4-p4-clean
```

STATUS PROVJERE

Ovaj primjer traži jezgru s modulom `8021q` i bridge podrškom za `vlan_filtering`. Redovna Ubuntu jezgra ih ima; u WSL2 treba provjeriti naredbom `ip link add link eth0 name eth0.10 type vlan id 10`. Primjer je napisan prema bilješkama i provjeren u obliku ispisa naredbi (`DRY=1`), ali **nije izvršen do kraja** u razvojnom okruženju (jezgra bez te podrške), pa tablica očekivanja iznad nije potvrđena izlazom. Ako nešto ne radi, prvo pokrenuti `make s4-p4` i javiti asistentu.

---

6 · PRIMJER 5 · S4/P5

Vatrozid iz podataka: Ansible rola, bastion i SSH

POZADINA PRIMJERA

Ručno pisanje pravila ne skalira: dodavanje nove usluge znači izmjenu više redova na više mjesta. Ovdje se politika opisuje **podacima** (koji segment smije kojem, na kojem portu), a šablon ih pretvara u `nftables`. Tako se pravila pregledaju kao podaci (koji je segment dobio koji pristup), a generisanje je jedno mjesto koje se testira. Rola koristi alate iz sedmice 3: šablon (Jinja2), handler i `validate`, kojim se izgenerisana datoteka provjerava *prije* nego što zamijeni radnu.

yaml — Isječak 10 · s4/p5/group_vars/rtr/vars.yml

```yaml
# Politika mreže kao PODACI: pravila se mijenjaju ovdje, a šablon ih pretvara u nftables.
fw_uplink: r-up
fw_vpn_if: wg-rtr
fw_vpn_port: 51820

fw_segmenti:                    # ime: podmreža
  web: 10.40.1.0/24
  app: 10.40.2.0/24
  db: 10.40.3.0/24

fw_javno:                       # internet -> segment (samo navedeni portovi)
  - { do: web, proto: tcp, portovi: [80, 443] }

fw_pravila:                     # segment -> segment (slojevi: web -> app -> db)
  - { od: web, do: app, proto: tcp, portovi: [8080] }
  - { od: app, do: db,  proto: tcp, portovi: [5432] }

fw_bastion: 10.40.1.10
# jedina adresa s koje je dozvoljen SSH admin pristup app i db slojevima
fw_admin:
  - { do: app, proto: tcp, portovi: [22] }
  - { do: db,  proto: tcp, portovi: [22] }

fw_izlaz: [web, app]            # segmenti kojima je dozvoljen izlaz prema internetu (db nije)
fw_vpn:                         # VPN klijent -> segment
  - { do: app, proto: tcp, portovi: [22, 8080] }

# Pravila se učitavaju u namespace rutera iz s4/p1; na pravom ruteru ostaje samo "nft".
fw_nft_cmd: ip netns exec ruo-rtr nft
fw_dest: /etc/nftables.d/ruo.nft
fw_instaliraj_paket: true
```

```
# SSH klijent: unutrašnje mašine su dostupne samo preko bastiona (ProxyJump)
ssh_korisnik: student
ssh_hostovi:
  app1: 10.40.2.2
  db1: 10.40.3.2
```

jinja2 Isječak 11 · s4/p5/roles/firewall/templates/ruo.nft.j2 (izvod)

```
{% macro portovi(l) %}{% if l | length == 1 %}{{ l[0] }}{% else %}{ {{ l | join(', ') }} }{%
endif %}{% endmacro %}
table inet ruo
delete table inet ruo

table inet ruo {
  chain forward {
    type filter hook forward priority filter; policy drop;

    ct state established,related accept
    ct state invalid drop

{% for r in fw_javno %}
    # internet -> {{ r.do }}
    iifname "{{ fw_uplink }}" ip daddr {{ fw_segmenti[r.do] }} {{ r.proto }} dport
{{ portovi(r.portovi) }} accept
{% endfor %}
{% for r in fw_pravila %}
    # {{ r.od }} -> {{ r.do }}
    ip saddr {{ fw_segmenti[r.od] }} ip daddr {{ fw_segmenti[r.do] }} {{ r.proto }} dport
{{ portovi(r.portovi) }} accept
{% endfor %}
{% for r in fw_admin %}
    # bastion -> {{ r.do }} (administracija)
    ip saddr {{ fw_bastion }} ip daddr {{ fw_segmenti[r.do] }} {{ r.proto }} dport
{{ portovi(r.portovi) }} accept
{% endfor %}
{% for s in fw_izlaz %}
    # izlaz: {{ s }} -> internet
    ip saddr {{ fw_segmenti[s] }} oifname "{{ fw_uplink }}" accept
{% endfor %}
{% for r in fw_vpn %}
    # VPN -> {{ r.do }}
    iifname "{{ fw_vpn_if }}" ip daddr {{ fw_segmenti[r.do] }} {{ r.proto }} dport
{{ portovi(r.portovi) }} accept
{% endfor %}

    limit rate 5/minute log prefix "ruo-drop: "
  }
```

yaml — Isječak 12 · s4/p5/roles/firewall/tasks/main.yml

```yaml
- name: nftables je instaliran
  ansible.builtin.apt: { name: nftables, state: present, cache_valid_time: 3600 }
  when: fw_instaliraj_paket

- name: direktorij za pravila
  ansible.builtin.file: { path: "{{ fw_dest | dirname }}", state: directory, mode: "0755" }

- name: pravila iz šablona (provjera sintakse PRIJE zamjene datoteke)
  ansible.builtin.template:
    src: ruo.nft.j2
    dest: "{{ fw_dest }}"
    mode: "0644"
    validate: nft -c -f %s          # neispravna datoteka nikad ne zamijeni radnu
  notify: učitaj vatrozid
```

nft — Isječak 13 · izgenerisano: /etc/nftables.d/ruo.nft (izvod)

```nft
    ct state established,related accept
    ct state invalid drop

    # internet -> web
    iifname "r-up" ip daddr 10.40.1.0/24 tcp dport { 80, 443 } accept
    # web -> app
    ip saddr 10.40.1.0/24 ip daddr 10.40.2.0/24 tcp dport 8080 accept
    # app -> db
    ip saddr 10.40.2.0/24 ip daddr 10.40.3.0/24 tcp dport 5432 accept
    # bastion -> app (administracija)
    ip saddr 10.40.1.10 ip daddr 10.40.2.0/24 tcp dport 22 accept
    # bastion -> db (administracija)
    ip saddr 10.40.1.10 ip daddr 10.40.3.0/24 tcp dport 22 accept
    # izlaz: web -> internet
    ip saddr 10.40.1.0/24 oifname "r-up" accept
    # izlaz: app -> internet
```

bash — Isječak 14 · pokretanje

```bash
make s4-p5           # syntax-check i --check --diff (bez izmjena)
make s4-p5-apply     # opcionalno: primjena dva puta, pa testovi (traži s4-p1-up)
```

izlaz 1. izvršavanja (PLAY RECAP)

```
localhost : ok=6 changed=3 unreachable=0 failed=0 skipped=1 rescued=0 ignored=0
```

izlaz 2. izvršavanja: changed=0 = idempotentnost

```
localhost : ok=5 changed=0 unreachable=0 failed=0 skipped=1 rescued=0 ignored=0
```

Pravila koja je rola izgenerisala daju isti ishod kao ručno napisana datoteka iz Primjera 2: matrica povezivosti ponovo daje deset od deset. Politika se sada mijenja u `vars.yml`, a ne u pravilima.

Bastion host

Administrativni pristup (SSH) ne treba biti otvoren prema svim mašinama. **Bastion** je jedna dobro čuvana mašina s koje je SSH prema app i db dozvoljen; sve ostale adrese su blokirane. U podacima je to

`fw_bastion` i lista `fw_admin`, a u vatrozidu pravilo s izvorom `ip saddr 10.40.1.10`. Klijent do unutrašnjih mašina dolazi kroz bastion opcijom `ProxyJump`, bez da je ijedna od njih izravno dostupna:

sshconfig — Isječak 15 · izgenerisano: ~/.ssh/ruo-config

```
# Ansible managed
# Upotreba: ssh -F ~/.ssh/ruo-config app1
Host bastion
  HostName 10.40.1.10
  User student

Host app1
  HostName 10.40.2.2
  User student
  ProxyJump bastion

Host db1
  HostName 10.40.3.2
  User student
  ProxyJump bastion
```

izlaz s4/p5/test-bastion.sh (izvorna adresa mijenja ishod)

```
  SSH iz 10.40.1.10 -> 10.40.2.2  :22  dozvoljeno  (očekivano: dozvoljeno) ok
  SSH iz 10.40.1.10 -> 10.40.3.2  :22  dozvoljeno  (očekivano: dozvoljeno) ok
  SSH iz 10.40.1.2  -> 10.40.2.2  :22  blokirano   (očekivano: blokirano ) ok
  SSH iz 10.40.1.2  -> 10.40.3.2  :22  blokirano   (očekivano: blokirano ) ok
```

BASTION NIJE DOVOLJAN SAM PO SEBI

Pravilo po izvornoj adresi štiti samo dok je adresa vjerodostojna. Bastion se dodatno štiti jakom autentifikacijom (SSH ključevi, bez lozinki), evidencijom pristupa i što manjim brojem usluga na njemu. U ovom labu je bastion samo dodatna adresa na web mašini, radi jednostavnosti; u stvarnom sistemu to je zasebna mašina.

SAŽETAK I PROVJERA ZNANJA

Pregled naredbi, pitanja i zadaci

CILJ	ŠTA RADI	MIJENJA SISTEM?
`make s4-p1`	ispis naredbi za lab (`DRY=1`)	ne
`make s4-p1-up`	ruter i segmenti web/app/db (namespace-ovi)	da (samo namespace-ovi, sudo)
`make s4-p2`	`nft -c`: provjera sintakse	ne
`make s4-p2-apply`	učitava vatrozid, matrica PRIJE/POSLIJE	da (u ruteru laba)
`make s4-p3` / `s4-p3-up`	ispis naredbi / WireGuard tunel + test	ne / da
`make s4-p4` / `s4-p4-up`	ispis naredbi / bridge + VLAN + test	ne / da (traži 8021q)
`make s4-p5` / `s4-p5-apply`	rola vatrozida: `--check` / dva izvršavanja + testovi	ne / da
`make s4-clean`	briše sve iz sedmice 4	uklanja

PITANJE	ODGOVOR
Zašto ruter bez `ip_forward=1` ne prosljeđuje pakete?	Jezgra zadano odbacuje pakete koji nisu za nju; prosljeđivanje se mora uključiti, čime je ruter svjesna odluka, a ne nuspojava.
Čemu služi `ct state established,related accept`?	Propušta odgovore na već dozvoljene veze, pa se pravilima opisuje samo smjer uspostave veze.
Zašto datoteka počinje s `table` pa `delete table`?	Učitavanje počinje od čistog stanja i ne javlja grešku ako tabela ne postoji, pa je idempotentno.
Zašto jedna tabela s `accept` ne otvara ono što druga zabranjuje?	Paket mora proći sve lance na istom hooku; `accept` završava samo svoj lanac.
Šta je `allowed-ips` u WireGuardu?	Istovremeno tabela usmjeravanja (šta ide kroz tunel) i kontrola pristupa (šta se prihvata od peera).
Po čemu se VLAN razlikuje od podmreže?	VLAN je L2 domena (ko dijeli okvire), podmreža je L3 adresni opseg; obično su 1:1, ali nisu isto.
Čemu `validate` u zadatku `template`?	Neispravna datoteka se provjeri prije zamjene radne, pa greška u šablonu ne ruši vatrozid.

Zadaci za samostalni rad

1. U `firewall.nft` dodati pravilo koje dozvoljava `web` prema `app` na port 9090, učitati ga i u `test.sh` dodati dvije provjere (dozvoljeno 9090, blokirano 9091). Objasniti zašto bez pravila veza ne radi.
2. Privremeno ukloniti `ct state established,related accept` i pokrenuti test. Objasniti šta se desi s odgovorima i zašto se pravilo vraća.
3. U `vars.yml` dodati segment `cache` (`10.40.4.0/24`) i pravilo `app` → `cache` na port 6379 te segment dodati u `SEGMENTI` u `lab.sh`. Pokrenuti rolu dva puta i predati izlaz drugog izvršavanja.

4. U `wg.sh` dodati drugog klijenta koji kroz tunel smije dosegnuti samo `web` segment (`allowed-ips` + pravilo za `wg` sučelje). Dokazati testom da ne doseže `app`.
5. Svakom pravilu u lancu `forward` dodati `counter`, pokrenuti test i objasniti koji brojač je porastao pri kojoj provjeri.
6. U `vlan.sh` dodati treći VLAN (30) s jednom mašinom i proširiti test izolacije (zahtijeva jezgru s VLAN podrškom).

> **SLJEDEĆA SEDMICA**
> Mreža i vatrozid su definisani kodom. Sedmica 5 uvodi imenovanje i komunikaciju: DNS zone i zapise (BIND/CoreDNS, Terraform DNS provideri), e-poštu (Postfix, Dovecot, SPF, DKIM, DMARC, TLS/ACME) te SFTP s chroot korisnicima iza vatrozida.
